Webtoon Project — GitHub & Firebase 복습 노트

2026-08-02 · 오늘 진행한 작업을 처음 배우는 사람 눈높이로 정리 · 복습용

0. 오늘 한 일 한눈에 보기

- GitHub 저장소 링크 확인 — 프로젝트 전체 코드/이력이 담긴 온라인 창고 주소
- 수집한 CSV 5종을 Firebase Firestore(문서형 데이터베이스)에 업로드
- Firebase Hosting에 대시보드(public 폴더)를 배포 — 실제 라이브 웹사이트에 반영
- Firebase 콘솔의 두 화면(프로젝트 개요 vs Firestore 데이터 화면) 차이 정리

1. GitHub란?

GitHub는 코드의 변경 이력을 git이라는 도구로 기록하고, 그 저장소(repository)를 인터넷에 올려 공유할 수 있게 해주는 서비스다. 로컬 컴퓨터의 프로젝트 폴더가 origin이라는 이름으로 GitHub 주소와 연결되어 있으면, git이 커밋한 내용을 그 주소로 밀어 넣거나(push) 받아올(pull) 수 있다.

포트폴리오·이력서·노션 등에 "이 프로젝트 다 보여줄게" 할 때 GitHub 링크 하나만 주면 코드, 커밋 히스토리(작업 순서와 이유), README까지 한 번에 보여줄 수 있어서 널리 쓰인다.

> git remote -v 로 현재 연결된 저장소 주소를 확인할 수 있음 (오늘 확인한 주소: github.com/kjyilj0707/webtoon_project)

2. Firebase는 하나가 아니라 여러 서비스의 묶음

"Firebase에 올린다"는 말은 사실 뭉뚱그린 표현이다. Firebase 안에는 성격이 전혀 다른 서비스들이 있고, 오늘은 그중 세 가지가 등장했다.

서비스	정체	오늘 쓰임
Firestore	문서(JSON) 단위로 저장하는 NoSQL 데이터베이스	CSV 5개를 컬렉션으로 변환해 저장
Storage	이미지, PDF, CSV 등 원본 파일 그대로 저장하는 공간	콘솔에서 미활성화 상태라 이번엔 사용 못함
Hosting	public 폴더의 HTML/JS/CSS를 인터넷에 정적 웹사이트로 배포	대시보드(app.js 등) 최신 버전을 라이브 반영

> 핵심: CSV "파일 자체"를 그대로 올리고 싶으면 Storage가 맞는 서비스지만, 오늘은 Storage가 콘솔에서 활성화되어 있지 않았다(유료 요금제 필요할 수 있음). 그래서 CSV의 "내용(행 데이터)"을 읽어서 Firestore에 문서로 바꿔 저장하는 방식을 택했다.

3. Firestore 핵심 개념 3가지

3-1. 컬렉션 / 문서 / 필드

Firestore는 엑셀과 비슷하게 생각하면 쉽다. 컬렉션(collection)은 시트(예: "webtoon_raw" 시트), 문서(document)는 그 시트의 한 행(웹툰 한 작품), 필드(field)는 그 행의 각 칸(titleName, favoriteCount 등)에 대응한다.

3-2. 문서 ID와 중복 데이터

문서마다 고유한 이름(ID)이 필요하다. webtoon_clean.csv는 titleId가 중복 없이 깨끗해서 titleId를 그대로 문서 ID로 썼다. 반면 webtoon_raw 등 원본 CSV는 같은 titleId가 여러 번 나오는 경우가 있어서, titleId 뒤에 행 번호를 붙여(예: 12345_87) 겹치지 않게 만들었다.

3-3. 배치 쓰기(batch)와 500건 제한

문서를 하나씩 저장하면 4천 건 넘는 데이터는 매우 느리다. Firestore는 한 번의 배치(batch)에 최대 500건까지 묶어서 보낼 수 있게 해준다. 그래서 오늘 스크립트는 400건마다 batch.commit()으로 한 번씩 끊어 보내는 방식을 썼다(여유를 두기 위해 500이 아닌 400 사용).

추가로 CSV의 빈 칸은 pandas에서 NaN(숫자도 문자도 아닌 특수 값)으로 읽히는데, Firestore는 NaN을 저장하지 못해 에러가 난다. 그래서 업로드 전 NaN을 None(값 없음)으로 바꿔주는 처리를 거쳤다.

4. 두 종류의 "열쇠" — 헷갈리기 쉬운 개념

	serviceAccountKey.json	firebase-config.js의 apiKey
누가 쓰나	내 컴퓨터(서버/스크립트)	브라우저(방문자 누구나)
권한	데이터를 마음대로 읽고 쓸 수 있는 강력한 관리자 키	"이 앱이 이 프로젝트 것"이라는 식별용, 그 자체론 힘이 약함
공개해도 되나	절대 안됨 → .gitignore로 git에 안 올라가게 막음	코드에 그대로 보여도 안전 (진짜 방어는 보안 규칙이 담당)

5. 오늘 실제로 만든 데이터 흐름

이 프로젝트는 웹툰 정보를 크롤링(수집) → 정제 → 분석하는 파이프라인이고, 각 단계 결과가 CSV로 남아있었다. 오늘은 이 CSV들을 각각 별도의 Firestore 컬렉션으로 옮겼다.

CSV 파일	→ Firestore 컬렉션	문서 수	의미
webtoon_raw.csv	webtoon_raw	4,103	목록 크롤링 직후 원본
webtoon_detail_temp.csv	webtoon_detail_temp	3,774	상세정보 수집 중간 저장본
webtoon_detail.csv	webtoon_detail	3,872	상세정보 수집 완료본
webtoon_errors.csv	webtoon_errors	231	수집 실패 기록
webtoon_clean.csv	webtoons (기존)	3,846	정제 완료, 대시보드가 참조하는 최종본

왜 최종본(webtoons) 하나만 놔두지 않고 원본·에러까지 다 올렸을까? 나중에 "정제 과정에서 얼마나, 왜 데이터가 걸러졌는지"를 다시 확인하거나 재현하고 싶을 때 원본이 있어야 하기 때문이다.

6. "배포(deploy)"란 무엇인가

로컬에서 public/app.js 같은 파일을 수정해도, 인터넷에 떠 있는 실제 사이트(https://wtproject-5a00f.web.app)는 자동으로 바뀌지 않는다. 로컬 변경사항과 라이브 사이트는 완전히 분리되어 있다.

firebase deploy --only hosting 명령을 실행해야, 로컬 public 폴더의 최신 파일들을 Firebase의 서버(CDN)에 업로드해서 "지금부터 이 버전을 보여줘"라고 반영시킨다. 오늘 이 명령으로 "전체 불러오기" 버튼이 추가된 최신 대시보드가 실제로 라이브에 올라갔다.

--only hosting 은 firebase.json에 등록된 여러 서비스(firestore 규칙 등) 중 Hosting만 배포하라는 뜻

7. Firebase 콘솔 화면 구분 (오늘 질문했던 부분)

- 프로젝트 콘솔(.../overview) — 건물의 로비. Firestore·Hosting·Storage 등 이 프로젝트에 붙은 모든 서비스 목록과 요약이 보이는 첫 화면.
- Firestore 콘솔(.../firestore/data) — 로비에서 들어간 특정 사무실. 실제 컬렉션·문서·필드 값을 표로 직접 확인하는 화면.

> 데이터 값 자체를 보고 싶으면 2번(firestore/data) 링크로 바로 들어가는 게 빠르다.

8. 용어 정리 (Glossary)

용어	쉬운 설명
repository (저장소)	코드와 변경 이력을 담은 폴더 전체
commit	"여기까지 저장" 하고 남기는 변경 이력의 한 단위
origin / remote	내 로컬 저장소와 연결된 GitHub 상의 주소
collection	Firestore에서 문서들을 묶는 단위 (엑셀 시트에 비유)
document	Firestore의 저장 단위 하나 (엑셀의 한 행)
batch write	여러 문서를 한 번에 묶어서 저장하는 방식(최대 500건)
NaN	pandas가 빈 칸/결측치를 표현하는 특수 값, DB 저장 전 변환 필요
service account key	서버·스크립트가 프로젝트에 관리자 권한으로 접근할 때 쓰는 비밀 키 파일
deploy	로컬 변경사항을 실제 서비스(라이브 서버)에 반영하는 행위
Hosting	정적 웹페이지 파일을 인터넷에 올려 서비스하는 Firebase 기능

9. 다음에 이어서 보면 좋은 것

- Firestore 보안 규칙(firestore.rules) — 지금은 테스트 모드일 가능성, 실서비스 전엔 반드시 학습
- Firebase Storage 활성화 방법과 콘솔 요금제(Blaze) 차이
- GitHub Actions 등으로 firebase deploy를 자동화하는 CI/CD 개념